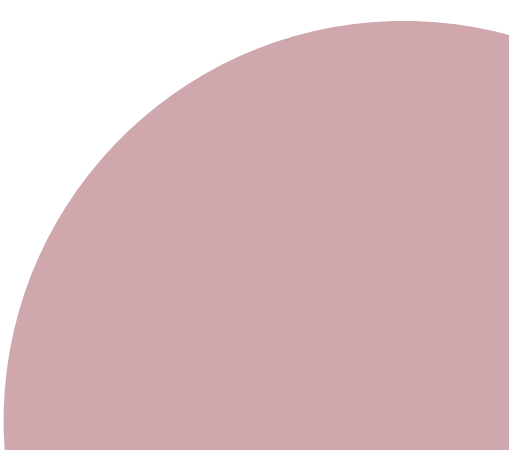


ML과 LLM의 작동 원리

인공지능 기초 심화탐구 보고서

2025-11-10

황태준
중앙고등학교
2025
2학년 7반 31번
지도교사 인기초 김화해 선생님



목차

I. 인공 신경망	4
I.1. 인공 신경망의 수학적 구조	4
I.2. 인공 신경망의 학습 방법	8
지도학습	8
비용함수	8
경사하강법 개요	10
경사하강법 수식 전개	11
경사하강법 계산	14
I.3. 인공 신경망 직접 구현해보기	15
I.4. 1부 참고자료 및 출처	15
II. LLM과 트랜스포머	16
II.1. 개요	16
II.2. 트랜스포머	18
II.3. 단어 임베딩	20
II.4. 어텐션	22
II.5. 임베딩과 어텐션의 작동 원리	23
II.6. 트랜스포머의 한계	27
II.7. 2부 참고자료 및 출처	28

이 보고서를 보여달라는 친구들이 좀 있어서 글이 설명문 형식으로 되어 진행이 부진할 수 있는 점 양해 부탁드립니다.

Abstract

인공지능은 말그대로 인간이 컴퓨터로 만든 지능이다. 그런데, 지능이란 무엇인가? 지능이 있다는 것은 생각할 수 있다는 것인가? AI는 생각하는가? 겉으로 보기에는 윤리적 판단도 어느 정도 하고, 수학 문제도 풀어내고, 각종 사소한 건에 대해 물어보면 나름 합리적인 판단도 내려주므로 생각하는 것처럼 보이기도 한다. 하지만 중학교에서 알고리즘을 처음 배울 때, 컴퓨터는 일정한 형식의 명령으로 계산(computation)만 빠르게 할 수 있는 바보라고 하지 않았는가? 구체적이고 특정적이지 않은, 모호한 인간의 자연어를 인공지능은 어떻게 처리하고 이해하여 그로부터 양질의 출력을 제공하는가?

지능(intelligence)의 정의는 아직도 제대로 내려지지 못하고 있다. 하지만 여러 정의에서 공통적으로 언급되는 부분은, 정보나 기술을 획득하여 목표 성취에 적용하고 새로운 것을 창조한다는 부분이다. 이 과정에서는 학습이 필수적이다. 정보나 기술을 획득하여 저장하고 이해(?)하는 과정이 학습이기 때문이다. 지능을 어떻게 정의 내리냐에 따라 컴퓨터가 지능을 가질 수 있는지 없는지는 철학적인 문제가 되겠지만, 적어도 지능을 모방할 수는 있을 것이다.

인공 신경망은 이러한 발상에서 비롯한 기술이자 자료구조로, 일종의 그래프이다. 정점은 뉴런의 신경세포¹를, 간선은 축삭을 본따 그 역할을 대신한다고 하는데, 생명과학을 배운 입장에서 이게 정말 맞는 말은 아닌 것 같고, 일반인들의 이해를 돕기 위한 비유적 설명으로 받아들일 수 있을 것이다.

정점들은 계층별로 모여 있는데, 직접적인 입력이 들어오는 입력 계층, 실질적인 처리가 일어나는 은닉 계층, 출력이 일어나는 출력 계층으로 나뉜다. 입력 계층과 출력 계층은 하나이며, 은닉 계층은 개수 제한은 없으나 그 개수와 해당 계층에서의 정점 개수를 어떻게 하느냐에 따라 학습 결과 성적이 좌우된다. 각 계층 간 정점은 수많은 간선들이 연결하고 있다. 최적화 알고리즘에서 모식적 모형으로 쓰이는 것들과 비슷하게, 여러 정점으로 가는 각 간선에는 서로 다른 가중치가 부여되어 있다. 인공 신경망을 통한 학습의 목적은, 궁극적으로 이 가중치들을 적절히 조절하여 주어진 입력에서 알맞은 출력을 내는 것이다. 하지만 수많은 간선들에 대해 일일이 수동으로 가중치를 설정할 수 없으니, 원하는 결과로부터 가장 가까

¹그냥 뉴런 자체를 의미한다고 하기도 한다.

은 출력을 내도록 역추적을 하는 방법을 사용하여 자동으로 학습을 진행한다. 이 방법을 역전파(backpropagation)이라고 하고, 이를 실현하기 위해 다변수 미적분학과 선형대수학의 손을 빌린 수학적 아이디어를 경사하강법(gradient descent)라고 한다.

하지만 인공 신경망은 생각보다 오래된 기술이고, 단점이 명확하다. 연산의 효율성은 둘째치고, 인공 신경망은 지능이 아니라 특정 목적 달성을 위해서만 단련된 시스템이기 때문에, 예를 들어 개와 고양이 사진을 구별하는 인공 신경망에 사람의 사진을 넣으면 무조건 틀린 답을 내놓게 된다. 사람은 개도 고양이도 아무것도 아닌 것²도 아니기 때문이다. 인공 신경망은 그 태생적 한계 때문에, 범용적이 될 수 없다.

현재는 거대 언어 모델(LLM)³의 시대이다. 물론 기초적인 형태의 언어 모델은 특정 목적을 위해서만 만들어지기도 했지만, 구글의 ‘트랜스포머’ 모델 개발 이후 문장 전체의 문맥을 파악할 수 있게 된 언어 모델은 급격히 범용 인공지능을 향해 달려나가 발전하고 있다. 지금 유명한 모든 또는 거의 모든 인공지능 언어 모델은 트랜스포머에 기반한다.

I. 인공 신경망

I.1. 인공 신경망의 수학적 구조

인공 신경망은 뇌의 구조에서 영감을 받았다고 하며, 실제 신경망을 모식적으로 본따 만든 그래프의 일종이다. 정점은 뉴런(neuron)이라고 부르며, 어떤 값을 나타내는 변수와 다른 계층의 정점을 가리키는 간선(또는 포인터)로 되어 있다. 각 뉴런이 가질 수 있는 값의 범위는 대개 실수 0에서 1이라던가(이런 걸 로짓[logit]이라고 한다), 정수 0에서 255라던가(16진수 RGBA 등) 하는 식으로 목적에 알맞게 임의로 정하지만 대개 관행을 따른다. 각 정점을 연결하는 간선에는 가중치가 부여되어 있으며, 출발 정점의 값이 간선을 거쳐 도착 정점에 값에 얼마나 영향을 주는지를 정하는 값이다.

이러한 계층 구조의 연속으로, 입력 계층에서 입력 데이터를 받고 은닉 계층들에서 정해진 규칙에 따라 선형적 가중치 합(weighted sum) 연산을 통해 데이터를 처리한 뒤 출력 계층으로 결과를 내놓는 것이 학습된 인공 신경망이 궁극적으로 수행해야 할 작업이다. 여기서 계층을 뚫으로써 분류할 데이터가 가지는 특성을 점점 세분화하여 구분하게 될 수 있는데, 바로 후술하겠으나 그 ‘세분화’라는 것이 우리의 눈에 합리적으로 보이지 않을지도 모른다.

²default 또는 empty 출력 정점

³대규모 언어 모델? 그냥 편하고 엘엘엠이라고 하겠습니다.

인공 신경망의 구조적 합리성은 사실 그리 직관적으로 와닿지 않는다. 어떻게 가중치들만 모아놓은 것이 모호한 데이터를 분류할 수 있게 하는 것이고, 도대체 각 정점에 할당된 값과 그것을 잇는 간선들의 가중치는 각각 무엇을 의미하는 것인지, 알기가 어렵다. 사실 인간이 수동으로 설계한 인공 신경망이 아닌 이상 학습 또한 결과 지향적으로 이루어지기 때문에 대부분의 경우 각 정점이 맡고 있는 바를 인간의 시각으로 보았을 때 논리적으로 합리적이 지 않을 가능성이 높다.

조금 모호한데, 인공 신경망을 수학적으로 좀 더 엄밀하게 살펴보자. 먼저, 입력 계층을 제외한 계층에 속한 어떤 정점 N 에 연결된 간선의 개수를 n 개라고 하자. 이때 간선이 연결된 정점의 값을 a_i , 간선의 가중치 값을 w_i 라고 한다면 N 의 값은 이전 정점과 간선들의 가중치에 의해서 정해지게 되는데, 직관적으로 선형 가중치 합인

$$\sum_{i=0}^n w_i a_i \quad (1.1)$$

가 된다. 이 값은 다시 다음 간선들의 가중치로 들어가 그 다음 계층의 정점들의 값에 영향을 주게 되는 연쇄적 반응을 일으킨다.

그런데, 정점에 할당될 수 있는 값의 범위는 정해져 있었지만 가중치 합으로 도출한 값은 어떤 값이든 될 수 있으므로 이 범위에 맞춰 주어야 하는데, 이때 활성화 함수를 사용한다. 대표적인 잘 알려진 활성화 함수로는 시그모이드가 있다. 시그모이드는 특정한 함수를 가리키는 것이 아니라, 두 점근선을 사이로 기울어진 S자의 개형을 띠는 곡선들을 모두 이르는 것이며, 생물군이나 경제 등의 성장을 기술하는 로지스틱 곡선이기도 한다. 시그모이드의 종류로는 오차(誤差)함수 $\operatorname{erf} x$, 역탄젠트 $\arctan x$, 쌍곡탄젠트 $\tanh x$, 구데르만 함수 $\operatorname{gd} x \equiv \int_0^x \operatorname{sech} t \, dt = \arctan(\sinh x)$ 등이 있으며, 별도의 스케일링(scaling) 조작 없이 0과 1 사이의 값을 가지는 아래 함수를 사용하기도 한다.

$$\frac{1}{1 + e^{-x}} =: \sigma \quad (1.2)$$

앞으로 별도의 설명 없이 시그모이드 함수라고 하면 이렇게 임의로 정의한 σ 함수⁴를 일컫는 것으로 한다. 아까의 정점 N 에 제대로 된(양식에 맞는) 값을 넣어 주려면, 앞선 가중치 합을 시그모이드 함수에 넣어야 한다.

⁴경우에 따라 이 표기는 표준편차와 헷갈릴 수 있다.

$$N = \sigma \left(\sum_{i=0}^n w_i a_i \right) \quad (1.3)$$

활성화 함수에는 시그모이드만 있는 것이 아니다. 사실, 시그모이드는 오래된 활성화 함수로 기울기 소실 문제⁵, 지수함수를 계산하기 위해 자원이 많이 드는 문제가 있다. 그래서 요즘은 ReLU(rectified linear unit) 등을 많이 사용한다. ReLU의 식은

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & (x \geq 0) \\ 0 & (x < 0) \end{cases} \quad (1.4)$$

로, 양수 부분에서 기울기가 사라지지 않으며 지수 연산이 없어 빠르다. 하지만 0 이하에서 기울기가 아예 없는데, 이것을 보완하기 위해 Leaky ReLU 같은 것도 있다.

$$\text{LReLU}(x) = \max(0.1x, x) \quad (1.5)$$

다만 ReLU 계열도 단점은 있는데, 여전히 중심이 0이 아닌 문제가 있다. 다른 활성화 함수로는 maxout이라고 불리는, 입력값의 최대치를 그대로 반환하는 함수나 ELU, softmax 등등이 있다.

한 정점에 영향을 주는 요인은 너무 많고, 얼마나 제대로 학습을 시키던 간에 결과가 완벽하게 한 출력 정점에 100%로 전달될 수 없다. 사실, 대부분의 모호한 경우 80%의 정확도를 얻는 것도 힘든 일이며, 은닉 계층에서는 더욱 이 현상이 심하다. 은닉 계층에서의 일종의 나비효과로 출력 계층의 값이 들쭉날쭉하지 않게 하기 위해 작은 값들은 필터링해버릴 수 있다. 우리는 선형적인 식을 다루고 있으므로, 필터링을 하기 위해서는 그냥 가중치 합에서 어떤 값을 빼버리면 된다. 특히 시그모이드는 가에서 기울기가 매우 완만하고 중앙 부분에서 가파르기 때문에, 값을 빼어서 그것이 충분히 작다면 거의 의미가 없는 값으로 변하여 초기 목적을 달성할 수 있게 된다. 이때 빼는 값을 편향(바이어스, bias)라고 한다. 구현할 때의 편의를 위해 수식으로 표현할 때는 음의 값을 더하는 식으로 생각하자.

$$N = \sigma \left(\sum_{i=0}^n w_i a_i + b \right) \quad (1.6)$$

이렇게 한 정점의 값을 정할 수 있다. 하지만 우리는 단순히 여러 개의 정점을 다루는 것이 아니라, 각 계층에 속한 여러 개의 정점을 다루고 있으며, 계층은 간선의 연결 여부를 정하므로 중요한 정보이다. 따라서 정점을 합리적으로 표기하려면 계층과 계층에서 정점의 위치를 모

⁵가에서 기울기가 0에 가까워서 기울기를 곱하는 연산을 시행하며 기울기가 점점 사라지는 문제

두 표현해야 한다. 계층을 L , 계층 내의 위치를 i 라고 한다면 한 인공 신경망 내의 어떤 정점은 $a_i^{(L)}$ 로 표현한다. 그러면 아까 한 정점을 정하는 식은 다음과 같다. $w_n^{(L)}$ 은 L 계층에서 오면서, 특정 정점에 연결된 것 중 n 번째 간선의 가중치를 의미한다.

$$a_j^{(L)} = \sigma \left(\sum_{i=0}^n w_i^{(L-1)} a_i^{(L-1)} + b_{L-1} \right) \quad (1.7)$$

모양새가 끔찍하다! 이렇게 해서는 시그모이드에 넣기도 전에 어지럼증으로 쓰러지고 말 것이다. 그래서, 모든 계산은 선형적이라는 것을 이용해 매개변수들을 행렬곱으로 나타낼 수 있을 것이다. 예를 들면 이런 식으로 말이다.

$$\begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ \vdots \\ a_n^{(1)} \end{bmatrix} = \sigma \left(\begin{bmatrix} w_0^{(0)} & w_1^{(0)} & \dots & w_n^{(0)} \\ w_0^{(1)} & w_1^{(1)} & \dots & w_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ w_0^{(k)} & w_1^{(k)} & \dots & w_n^{(k)} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0^{(0)} \\ b_1^{(0)} \\ \vdots \\ b_n^{(0)} \end{bmatrix} \right) \quad (1.8)$$

이제 가중치 행렬을 $\mathbf{W}$, 정점 벡터를 $\mathbf{a}$, 바이어스 벡터를 $\mathbf{b}$ 라고 한다면 식은 아주 간단해지게 된다.

$$\mathbf{a}^{(1)} = \sigma(\mathbf{W}^{(0)}\mathbf{a}^{(0)} + \mathbf{b}^{(0)}) \quad (1.9)$$

더 포괄적으로,

$$\mathbf{a}^{(L+1)} = \sigma(\mathbf{W}^{(L)}\mathbf{a}^{(L)} + \mathbf{b}^{(L)}) \quad (1.10)$$

실제로 이런 식으로 순전파(feed forwarding)⁶가 코드 상으로 구현되며, 행렬의 배열인 텐서가 정말 많이 사용된다. 모 머신러닝 라이브러리는 아예 이름부터 텐서가 들어가기도 한다. 아래는 순전파 연산 구현 예시이다.

```
import numpy as np
class FFNetwork:
    @staticmethod
    def sigmoid(x):
        return 1 / (1 + np.exp(-x));
    def __init__(self, weights, biases):
        self.weights = weights;
        self.biases = biases;
    def feedForwardUnit(self, a):
        for b, W in zip(self.weights, self.biases):
            a = sigmoid(np.dot(W, a) + b);
        return a;
# 신경망 구조 구현 코드...
```

⁶입력 계층으로 입력된 데이터가 여러 개의 은닉 계층을 거쳐서 출력 계층으로 값이 출력되는 과정을 순전파라고 한다.

I.2. 인공 신경망의 학습 방법

지도학습

인공 신경망은 수많은 계층과 그 속의 더 수많은 정점과 간선을 거쳐 결과를 도출한다. 따라서 엄청나게 많은 간선의 가중치를 사람이 하나하나 조작하는 것은 비효율적이다. 따라서 학습은 특정한 알고리즘에 의해 자동으로 진행되어야 한다. 초등학교, 중학교 때 한 번씩은 접해왔고 인공지능 기초 수업시간에도 했었던 티쳐블 머신이나 엔트리 머신러닝 등등은 라벨을 통한 지도학습의 예시이다. 이것은 신경망에게 어떤 데이터가 어디에 속해야 하는지 알려줌으로써 비슷한 경향을 가진 데이터를 분류해내는데 적합한 가중치 값들을 정할 수 있도록 한다. 이것이 인공 신경망에서 ‘학습’의 본질이다.

처음에 신경망의 가중치는 모두 정해진 범위 안에서 무작위로 설정된다. 이 무작위성은 그리 중요한 것은 아니고, 서로 각각 어느 정도 차이가 있는 값들로 되어 있어서 어떤 데이터를 나와도 모든 출력 계층의 정점이 대체로 균등하게 활성화되면 된다. 이렇게 해야 지도가 용이하기 때문이다.

지도를 하려면 먼저 어떤 라벨에 속하는 데이터를 신경망에 입력한다. 신경망은 어떤 값을 출력할 것인데, 그 값에 관계 없이 그 데이터가 어디에 속하는지 알려준다. 신경망은 해당 데이터가 입력되었을 때 제시된 라벨에 대응하는 값을 출력하도록 자신의 가중치들을 조정한다. 이것을 반복함으로써 학습이 이루어진다. 그렇다면, 신경망에게 어떻게 데이터가 어디에 속하는지 알려줄 것이며, 그에 따라 가중치 값은 어떻게 조정해야 하는 것일까? 여기서 비용함수 또는 손실함수(cost function, loss function)의 개념이 등장한다.

비용함수

비용함수는 $C(\hat{\mathbf{y}}, \mathbf{y})$ 와 같이 표기한다. $\hat{\mathbf{y}}$ 는 모델이 예측하는 값⁷, 즉 예측값의 벡터이고, $\mathbf{y}$ 는 실제값, 즉 정답의 벡터이다. 이 값이 작을수록 신경망 모델이 정확하다는 뜻이 된다. 여기서 비용함수의 이름이 나왔는데, loss(손실)는 말 그대로 정확도나 데이터 무결성 따위가 얼마나 손실되었느냐를 의미하고, cost(비용)은 이것을 최적화 문제의 관점으로 보아, 비용을 최소화하는 것을 목적으로 두었을 때 견해를 반영한 용어이다. 즉, 신경망의 학습은 사실상 가중치와 편향을 조정하여 비용함수의 값을 최소화하는 것이다.

$$\min_{\mathbf{w}, \mathbf{b}} C(\hat{\mathbf{y}}, \mathbf{y}) \quad (1.11)$$

⁷expected(expectation) value 가 아니라 predicted value이다. 즉 기댓값(평균)과는 다르다.

비용함수에는 여러 형태가 있는데, 대표적으로 평균제곱오차(MSE, mean squared error) 방식과 교차 엔트로피 방식이 있다.

먼저, MSE 꼴은 다음과 같다⁸.

$$C = \frac{1}{2}|\mathbf{y} - \hat{\mathbf{y}}|^2 = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y)^2 \quad (1.12)$$

참고로, MSE는 분산과 식 구조가 동일하지만 주목하는 대상이 다르다. 근본적으로 MSE는 $\mathbb{E}[(Y - \hat{Y})^2]$ 인데, 예측값 $\hat{Y}$ 를 기댓값 $\mathbb{E}(Y)$ 로 본다면(또는 두 값이 같다면) 이것은 분산 $\mathbb{E}[(Y - \mathbb{E}[Y])^2] = \mathbb{E}[Y^2] - (\mathbb{E}[Y])^2$ 과 같게 된다. 하지만 이것이 계산적인 측면에서 분산과 비슷한 의미를 가진다는 것을 실감하면 MSE가 의미하는 바를 어렵지 않게 알 수 있다. 이때, 차에 제곱을 취하므로, 정답보다 출력값이 멀면 멀수록 더 강한 페널티를 부여하게 된다.

교차 엔트로피(확률 분포) 꼴은 다음과 같다.

$$C = - \sum_{i=1}^n y_i \log \hat{y}_i \quad (1.13)$$

일반적으로 $\mathbf{y}$ 는

$$\mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \quad (1.14)$$

과 같이 하나만 1인 one-hot encoded 벡터이므로 이 경우 남는 항은

$$C = - \log \hat{y}_{\text{정답 index}} \quad (1.15)$$

교차 엔트로피 꼴은 기댓값의 꼴 $\sum_i x_i p_i$ 과 비슷하게 생겼는데, $\mathbf{y} \sim y_i$ 는 실제 정답의 분포, $\hat{\mathbf{y}} \sim \hat{y}_i$ 는 모델이 예측한 분포라는 통계적 관점에서 봤을 때 그 의미가 드러나는 것이다. $\log \hat{y}$ 는 모델이 정답이 $\hat{y}$ 라고 확신하는 정도(confidence)를 의미한다. 여기에 음수를 취함으로써, 예측값이 실제값에 가까워져서, 정답에 대한 확실도가 커질수록 비용함수는 작아지게 된다.

⁸벡터의 크기는 노름(norm) 표현 $\|\cdot\|$ 으로 쓰는 것이 좀 더 일반적이지만, 간단함을 위하여 절댓값 기호 $|\cdot|$ 로 통일하도록 하겠다.

분류에는 보통 교차 엔트로피 방식이 사용되나, 이 보고서에서 비용함수의 식이 추가로 등장하게 된다면 더 익숙하고, 무엇보다 대수함수라서 로우 레벨 언어로도 계산이 편리한 MSE로 가보도록 하겠다. MSE는 보통 분류보다는 회귀(regression), 실수값 예측 등에서 더 빈도 있게 사용되기는 한다.

경사하강법 개요

그렇다면 비용함수를 최소화할 방법은 무엇인가? 수학Ⅱ에서 배웠듯, 어떤 함수들은 도함수가 0인 지점에서 최솟값(global minimum) 또는 극솟값(Local minimum)을 가질 가능성이 있다. 즉, 임의의 매개변수 p 에 대해

$$\frac{dC}{dp} = 0 \Rightarrow \min C = C(p) \quad (1.16)$$

가 성립...할까?

여기에는 몇 가지 문제점이 있다. 먼저, 가장 근본적으로 도함수의 값이 0이라고 해서 그 점이 무조건 극소점이 아니다. 반례는 널리고 널렸다. 다음으로, 비용함수는 1차원이 아니다. 하나의 매개변수에 의해서만 결정되는 함수가 아니라는 말이다. 즉, 도함수가 0인 지점을 방정식을 풀어 직접 찾아내는 것은 그리 좋은 방법이 아니다.

이것보다는, 아무 점에서 시작하여 함숫값이 작아지는 방향으로 점을 옮겨가면서 마침내 도함수가 0이 되는 곳을 찾는 것이 더 포괄적이고 유연할 것이다. 특히, 현재 위치한 점에서 접선의 기울기를 알 수 있다면 어느 방향으로 가야할지를 쉽게 알 수 있다. 접선의 기울기의 크기에 비례하여 움직이는 정도를 조정한다면 더 효율적으로 할 수 있을 것이다. 이 과정을 반복하다보면 어떤 극소점에 도달하게 된다. 한 가지 주목할 것은, 극소점은 이런 식으로 쉽게 찾을 수 있지만 최소점은 정말 찾기가 어렵다. 극소점이 최소점이라는 보장이 없으며, 어디서 점을 시작해야 최소점으로 굴러가는지 알 수 없기 때문이다.

어쨌든, 이 규칙은 매개변수 공간이 몇 차원이던간에 똑같이 적용된다. 해당 공간에서 점이 어느 방향으로 이동해야 함숫값이 작아지겠는가에 대한 것이기 때문에 꼭 1차원일 필요가 없는 것이다. 이렇게, 변수가 여러개일 때 사용할 수 있는 것이 그라디언트(gradient)⁹ ∇C 이다. 그라디언트는 공간의 기울기를 나타내는 연산자로, 이 경우에는 어느 방향으로 가장 가파르게 증가하는지를 나타내는 벡터이다. 그러므로 $-\nabla C$ 는 어느 방향으로 가장 가파르게

⁹이 보고서에서 델 연산자에 대해 더 깊이 다루지는 않는다. 하지만 그라디언트의 수학적 특성에 대해서는 조금 뒤에 서술하겠다.

감소하는지를 알려주게 된다. 극소를 찾는 방법은 어떤 점에서의 그라디언트를 계산하고, 그라디언트의 반대 방향으로 조금 이동하는 것을 반복하는 것이다.

이렇게 그라디언트를 이용해 함수 곡면을 따라 내려가면서 극소를 찾는다는 의미로 이 방법을 gradient descent 라고 하고, 한국어로 적절히 번역해서 경사하강법이라고 한다.

경사하강법 수식 전개

구체적으로 어떻게 하는 것인지 알아보자. 먼저 모델이 예측을 하는 것을 수학적으로 나타낼 필요가 있다. $\hat{y}$ 라는 것은 x 를 입력했을 때, 가중치와 바이어스 θ 에 따라 신경망 f 를 통과해서 나온 출력이다. 즉, 다음과 같다.

$$\hat{y} = f(x, \theta) \quad (1.17)$$

그러므로

$$C(\hat{y}, y) = C(f(x, \theta), y) \quad (1.18)$$

이고, 지금 상황에서 입력 x 와 정답 y 는 정해진 것이므로 비용함수를 $C(\theta)$ 로 간단히 나타내도록 하자. 하지만 생략이 일어났으므로 명확히 하기 위해 변수의 종속 관계를 살펴보면,

$$\begin{aligned} \theta &\mapsto f(x, \theta) = \hat{y} \\ \hat{y}, y &\mapsto C(\hat{y}, y) \\ \Rightarrow \theta &\mapsto C(f(x, \theta), y) \\ \therefore C(\theta) &= (C \circ f)(x, \theta) \end{aligned} \quad (1.19)$$

모델을 학습시킬 때는 모델이 예측한 출력값이 아닌 가중치와 바이어스를 조작하는 것이므로, θ 에 주목할 필요가 있는 것이다. 이제 식을 유도하겠다. θ 는 모든 가중치와 바이어스의 벡터이고, 편의를 위해 앞으로 볼드 표기를 하지 않고 그냥 θ 로 쓰겠다. 목표는 아래를 구하는 것이다. d 는 입력 벡터의 길이 또는 입력 변수의 개수, 즉 차원 수이다.

$$\min_{\theta \in \mathbb{R}^d} C(\theta) \quad (1.20)$$

그라디언트 $\nabla C(\theta)$ 는 각 성분에 대한 편미분으로 이루어진 벡터이다.

$$\nabla C(\theta) = \begin{bmatrix} \frac{\partial C}{\partial \theta_1} \\ \vdots \\ \frac{\partial C}{\partial \theta_d} \end{bmatrix} \quad (1.21)$$

함수 C 에서 방향벡터 $\mathbf{u}$ 로의 순간변화율은 $\nabla C(\theta)^\top \mathbf{u}$ 이다¹⁰. 따라서 $\nabla C(\theta)$ 는 가장 가파르게 증가하는 방향이고, $-\nabla C(\theta)$ 는 가장 가파르게 감소하는 방향이다.

가장 단순한 형태의 경사하강법은 현재 위치에서 음의 그라디언트 방향으로 그라디언트의 크기에 비례해 이동하는 것이다.

$$\theta_{k+1} = \theta_k - \eta \nabla C(\theta_k) \quad (1.22)$$

여러 문헌에서 볼 수 있는 좀 더 일반적인 표기는¹¹ 다음과 같다. 이런 식을 업데이트 식이라고 하기도 한다.

$$\theta \leftarrow \theta - \eta \nabla C(\theta) \quad (1.23)$$

여기서 $\eta > 0$ 는 학습률이다. 식으로 보면 알 수 있지만, 학습률은 매개변수가 학습에 반영되는 정도를 결정한다. 학습률이 너무 낮으면 모델이 자기 수치를 조정하는 정도가 작아져 학습이 느리거나 덜 된다. 학습률이 너무 높으면 흔히 오버슈팅(overshooting)이라고 하는, 상대적으로 작은 매개변수에도 수치가 과잉 조정되는 현상이 일어날 수 있다. 이 현상은 모두 점이 극점에 도달하는 것을 방해한다.

기본적으로 C 는 $\mathbb{R}$ 에서 연속이고 미분 가능하다고 가정해야 뭔가를 할 수가 있다. 이 가정에 의해 립시츠(Lipschitz) 규칙¹² $\Delta f(x) \leq \Delta x \max f'(x)$ 가 성립한다. 따라서 ∇C 에 대해 아래도 성립하게 된다.

$$\forall \mathbf{x}, \mathbf{y} \exists L > 0 \quad |\nabla C(\mathbf{x}) - \nabla C(\mathbf{y})| \leq L|\mathbf{x} - \mathbf{y}| \quad (1.24)$$

립시츠 연속 특성과 테일러 전개에 의해 아래가 성립하게 된다. 앞으로 $\mathbf{x}, \mathbf{y}$ 는 x, y 로 쓰겠다.

$$\forall \mathbf{x}, \mathbf{y} \quad C(\mathbf{y}) \leq C(\mathbf{x}) + \nabla C(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2 \quad (1.25)$$

$\mathbf{y} = \mathbf{x} - \eta \nabla C(\mathbf{x})$ 를 대입하면,

¹⁰외적 연산을 풀어 쓴 것이다.

¹¹ $\leftarrow$ 는 수학적으로는 $=$ 과 비슷하지만, 우변을 좌변에 대입, 할당한다는 의미를 강조한 것이다. 가장 간단한 한 줄은 $a \leftarrow a + k$ 같은 것이 있겠으며, 코드로는 `a = a + k;`로 쓰는 느낌이다.

¹²실수 전체에서 연속이고 미분 가능한 함수는 접선의 기울기의 크기가 항상 어떤 상수보다 작다는 규칙. Lipschitz continuity 영문 위키백과 페이지에 단번에 이해가 되는 좋은 GIF가 있다.

$$\begin{aligned}
 C(x - \eta \nabla C(x)) &\leq C(x) + \nabla C(x)^\top (-\eta \nabla C(x)) + \frac{L}{2} |\eta \nabla C(x)|^2 \\
 &= C(x) - \eta |\nabla C(x)|^2 + \frac{L\eta^2}{2} |\nabla C(x)|^2 \\
 &= C(x) - \left(\eta - \frac{L\eta^2}{2} \right) |\nabla C(x)|^2
 \end{aligned} \tag{1.26}$$

따라서 $\eta \in \left(0, \frac{2}{L}\right)$ 이면 계수 $\eta - \frac{L\eta^2}{2} > 0$ 이므로 매 반복마다 C 가 감소하여 하강하게 된다. C 가 하강하지 않고 일정하게 유지될 수도 있는데, 이것은 극소점을 찾았다는 뜻이 된다. 특히, $\eta \leq \frac{1}{L}$ 일 때는

$$C(x - \eta \nabla C(x)) \leq C(x) - \frac{\eta}{2} |\nabla C(x)|^2 \tag{1.27}$$

즉 학습률이 합리적으로 작은 값인 상태에서 음의 그라디언트 방향으로 이동하면 C 의 값이 감소하거나 일정함을 보장한다. 또, 거의 자명하게 볼록한 함수와 볼록하지 않은 함수에서 그라디언트 하강 시 극소점으로 갈 수 있음을 증명할 수 있는데, 이것은 재미 없으므로 생략한다.

참고로, 다른 형태의 하강법도 존재한다. 대표적인 것으로 확률적 경사하강법(SGD, stochastic gradient descent)와 뉴턴 방법이 있다.

확률적 경사하강법은 기존의 경사하강법¹³이 전체 데이터셋을 입력받아 작동하기 때문에 느리고 자원을 많이 먹는다는 단점을 보완하기 위해 만든 방법으로, 데이터셋에서 임의의 샘플을 추출하고 그 샘플로만 경사하강법을 수행하는 것이다. 이 덕분에 자원을 덜 요구하고 학습 속도도 빠르다. 하지만 데이터의 수가 적다보니 불안정하게 하강하고 요동치게 된다. 최적해인 극소점에 근접하게는 도달하나, 극소점에 정확히 도착하지 못할 수도 있다. 이러한 문제를 해결하기 위해 미니배치 경사하강법(mini-BGD)가 등장했다. 데이터셋을 딱 나누어떨어지는 일정한 수의 배치로 나눈 다음, 각 배치에 대해 각각 경사하강법을 수행하고 평균을 내는 방식이다. 보통 SGD를 얘기하면 보통 이 mini-BGD를 말하는 것이다. 확률적 경사하강법의 업데이트 식은

$$\theta_{k+1} = \theta_k - \eta_k g_k \quad \text{where } \mathbb{E}[g_k] = \nabla C(\theta_k) \tag{1.28}$$

¹³여태까지 살펴본 경사하강법으로, BGD(batch gradient descent)라고 한다.

¹⁴자세히 다루진 않겠으나 이 내용이 실린 논문이 발표됨으로써 SGD의 기반이 다져졌다.

이때 수렴하기 위해 일반적으로 만족해야 하는 로빈스-먼로(Robbins-Monro) 조건¹⁴이 있다.

$$\sum_{k=0}^{\infty} \eta_k = \infty, \sum_{k=0}^{\infty} \eta_k^2 < \infty \quad (1.29)$$

뉴턴 방법은 뉴턴이 처음에 기계학습이 아닌 함수의 근, 즉 $f(x^*) = 0$ 이 되게 하는 x^* 값을 찾을 때의 해법으로 제시한 것으로, 자신이 발명한 미분을 적극 활용하여 아래와 같은 논리를 제시했었다.

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (1.30)$$

최적화 문제에서는 함수 값이 아니라 도함수 값이 0이 되게 하는 것이 목표이므로 1차원에서

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)} \quad (1.31)$$

처럼 되어야 하고, 우리의 경우 다변수를 다루고 있으므로 그라디언트와 함께 헤시안(Hessian)¹⁵을 사용한다.

$$\theta_{k+1} = \theta_k - \frac{\nabla C(\theta_k)}{H_C(\theta_k)} = \theta_k - H_C(\theta_k)^{-1} \nabla C(\theta_k) \quad (1.32)$$

지수적으로 최적해에 수렴하고, 더 정확한 결과를 보이지만 헤시안과 그 역행렬을 계산하는 과정이 시간과 자원을 많이 소모한다. 그러므로 헤시안을 정확히 구하는 것이 아니라 근사하는 방법(BFGS 등)을 사용하여 이 단점을 보완한다.

경사하강법 계산

모든 이론적인 부분이 완료되었으므로 이제 조금 더 구체적인 계산 부분으로 들어갈 수 있다. 어떤 입력 x 에 대해 마지막 계층에서의 순전파는

$$z = Wx + b, \quad a = \sigma(z) \quad (1.33)$$

라고 할 수 있으며¹⁶, MSE 비용함수 값은

¹⁵헤시안 또는 헤세 행렬은 스칼라 함수 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 의 이계도함수 행렬이다. 일계 도함수 행렬은 야코비안(Jacobian)으로, 벡터 함수 $\mathbf{F}: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 에 대응한다. 둘의 관계는 그라디언트와 라플라시안의 관계와 비슷하다.

¹⁶이제 무엇이 벡터이고 무엇이 스칼라인지 알 수 있으므로 그냥 쓰겠다. 실제로 여러 이산수학, 기계학습 또는 다른 분야에서도 혼동의 여지가 없다면(있어도) 벡터와 스칼라를 구분하지 않는 교재가 많다.

$$C = \frac{1}{2}|a - y|^2 = \frac{1}{2} \sum_i (a_i - y_i)^2 \quad (1.34)$$

그러면 합성함수의 미분(chain rule)에 의해

$$\begin{aligned} \frac{\partial C}{\partial z} &= \frac{\partial C}{\partial a} \odot \frac{\partial a}{\partial z} = (a - y) \odot \sigma'(z) \because C(a(z)) = \frac{1}{2}|\sigma(z) - y|^2, \\ \frac{\partial C}{\partial W} &= \frac{\partial C}{\partial z} \frac{\partial z}{\partial W} = \left(\frac{\partial C}{\partial z}\right) x^\top \because \frac{\partial z}{\partial W} = x^\top, \\ \frac{\partial C}{\partial b} &= \frac{\partial C}{\partial z} \frac{\partial z}{\partial b} = \frac{\partial C}{\partial z} \because \frac{\partial z}{\partial b} = I \end{aligned} \quad (1.35)$$

여기서 $\odot$ 은 원소별로 곱한다는 뜻이다. 이렇게 구한 값으로 경사하강을 하는 것이 역전파(back propagation)의 원리이며, 이를 통해 인공 신경망의 학습이 이루어진다.

I.3. 인공 신경망 직접 구현해보기

중학교 때 C로 연결리스트를 직접 만들던 것이 생각나서 순전파, 역전파로 학습하고 분류할 수 있는 간단한 인공 신경망을 구현하기로 했습니다. 파이썬으로 하는 게 정상이라던데 저는 사실 파이썬을 잘 못해서 익숙한 C++로하기로 했습니다. 중간에 막히는 부분이 있어서 GPT한테 물어보면 파이썬으로만 대답하는 게 원통하답니다. 그래도 알고리즘 짜는 측면으로의 도움은 좀 받을 수 있었습니다. 처음에는 MSE로 하려고 했으나 크로스 엔트로피가 해 보니까 성능도 좋고 오히려 더 간단했어서 노선을 틀었습니다.

목표는 8×8 이미지를 2차원 `std::vector<int>` (사실상 `bool`)로 입력받아서 무슨 모양인지 분류하는 신경망을 만드는 것입니다.

코드 내장 주석 외에 추가 설명은 하지 않습니다. 코드는 다음 Github 주소에 올라가 있습니다: https://github.com/tjwhang/CPPNeuralNetwork/blob/main/neural_network.cpp

I.4. 1부 참고자료 및 출처

- 여러 위키백과 영문 문서: 개념 정립 및 서술에 큰 도움을 주었습니다. 학술적 내용의 위키백과의 경우 해당 문서에서 제시하는 모든 서술에 주로 전공서나 논문으로 출처가 적혀있기 때문에 신뢰 가능하다고 볼 수 있겠습니다(설마 누가 수학적 사실로 장난질을..!).
- 3Blue1Brown의 Neural Networks 시리즈: 직접적인 내용을 적는 데는 도움이 별로 안 되었지만 내용 구성을 참고하여, 산발적으로 탐구한 내용을 정리할 때, 논리적 흐름을 전개해서 짜임 있는 보고서를 만드는 데 도움을 주었습니다.

- Polyak, B. (1987), "Introduction to Optimization"
- Akilov, G. P. Kantorovich, L. V.(1982). "Functional Analysis" 2/e
- Holmes, M. (2023), "Introduction to Scientific Computing and Data Analysis" 2/e
- IBM社の 학습 자료들. "What is gradient descent?", "What is learning rate?" 등등

II. LLM과 트랜스포머

II.1. 개요

LLM은 특히 요즘들어 정말로 생각을 하고 있는 것처럼 보이지만, 사실 그렇지 않다. 현재까지 기계학습의 본질은 크게 다르지 않기 때문이다. LLM은 방대한 학습 자료를 바탕으로 이전 단어(들)를 입력받아 다음 단어로 나올 확률(확실한 정도)가 가장 높은 것을 뽑는 프로그램이다. GPT와 같은 챗봇을 사용할 때는 모델이 자기가 뽑은 단어를 다시 입력해 반복적으로 한 단어씩 뽑는 과정이 반복되는 것이다. 챗봇 UI에서 단어가 타자치듯 생겨나거나 페이드인 되는 효과는 단순 연출이 아니라 모델이 한 번에 하나의 토큰을 예측하기 때문에 생기는 것이다. 따라서 LLM은 인공신경망으로 구현된 복잡한 하나의 매개변수 함수일 뿐이다.

LLM을 정확하게 만드는 방법은 단순하다. 언어 모델이 언어를 "정확"하게 구사하게 하려면 단순히 엄청난 양의 데이터를 때려넣으면 된다. LLM에는 인공신경망에서처럼 가중치가 있는데, 정말 수없이 많이 있다. 마찬가지로 처음에는 모든 가중치가 무작위의 값이다가, 데이터를 받아 학습하면서 알맞은 결과를 내도록 조정해 나가게 된다.

학습 방법은, 간단히 예를 들어 실제 문장에서 마지막 단어만 뺀 것을 입력한 뒤 출력되는 결과를 받아 가중치를 수정해 실제 문장의 마지막 단어를 출력할 수 있도록 한다. 이 방법에 대해서는 앞서 역전파와 경사하강법 부분에서 다루었다. 이것을 수천억번 반복하면 학습한 문장은 물론, 처음 보는 문장에 대해서도 더 합리적인 예측을 하게 된다. LLM을 학습시키는데는 엄청난 컴퓨팅 자원이 필요한데, 1초에 10억 번 사칙연산이 가능한 컴퓨터로 적당히 합리적인 문장을 뽑아내는 LLM을 만들기 위해서는 학습시키는데만 1억 년 정도가 걸린다고 한다. 그래서 이러한 막대한 연산을 할 수 있는 하드웨어인 GPU¹⁷가 부상하기 시작했다. 이제는 웬만한 GPU로도 부족하니 NPU, TPU 등 행렬곱 연산에 최적화된 맞춤형 하드웨어를 만들어내고 있다.

¹⁷고성능 GPU는 동시에 수많은 연산을 병렬로 하기 위해 연산 코어를 아주 많이 때려넣어놓은 연산 장치로, 게임 그래픽, 3D/동영상 렌더링, 유체 시뮬레이션 등을 목적으로 만들어졌다.

기본적으로 LLM은 아무 문장이나 받아서 합리적인 문장을 잇는 것을 잘하면 된다. 하지만 좋은 LLM과 좋은 AI 챗봇의 조건은 다르다. 사용자의 요구를 알아듣고, 사실에 기반해 대답하고, 일어나지 않은 일에 대해 단서를 가지고 추론할 수 있어야 한다. 이를 위해

RLHF(Reinforced Learning with Human Feedback, 인간 피드백을 통한 강화학습)을 수행한다. 사람들이 직접 모델의 응답을 평가해 비용 함수를 구성하여 인간 선호도에 맞게 응답을 조정하는 것이다. 이 과정에서 원치 않는 대답을 걸러내거나, 원하는 대답으로 유도하게 된다.

그런데, 사실 GPU가 주목받기 시작한 것은, LLM의 연산 자체를 병렬로 처리할 수 있게 된 트랜스포머 모델의 등장 때문이다. 이전까지는 LLM은 한 번에 한 단어만 읽어오며 문장을 입력받았다. 하지만 구글이 2017년 발표한 트랜스포머는 문장의 단어들을 한꺼번에 읽어온다.

모든 LLM은 결국 언어를 숫자로 바꾸어야 하는데, 이때 숫자는 자기 나름의 의미를 나타내는 값들로 표현된다¹⁸. 트랜스포머는 각 단어를 단순히 자기가 학습한 일반적인 의미에 따른 숫자로 변환하는 것뿐만 아니라, attention 이라는 과정을 수행해 문장 내에서 각 단어의 상호작용을 고려해 각 단어가 문맥적으로 어떤 의미를 가지는지도 파악할 수 있게 된다. 예를 들어 “배가 아프다”의 ‘배’와 “배를 타다”의 ‘배’는 다른 의미를 가지고 있는데, 트랜스포머는 문장 전체를 보아 각각이 어떤 의미인지를 명확히 알 수 있다는 뜻이다.

의미를 기반으로 수치 벡터를 형성하고 attention을 거친 트랜스포머는 일반적으로 결과 벡터를 사전 학습된 인공신경망에 순전파하는 과정을 거친다. 순전파를 하여 단순한 선형결합인 attention 출력 결과를 시그모이드나 ReLU 같은 활성화함수에 통과시켜 비선형적으로 만들어으로써 복잡한 표현을 조성한다. 즉, 언어적 능력(capacity)을 향상시킨다. 이 출력값을 다시 받아 attention을 수행하고 또 순전파를 하고... 하는 것을 충분히 반복한다. 이렇게 반복하는 과정에서 모델은 어떤 방향으로 단어를 사용해 대답을 내놓아야할지 결정한다. 마침내 마지막에는 입력 문장의 모든 단어에 영향을 받아 수치가 조정된 가장 마지막 단어에 해당하는 수치벡터를 바탕으로 다음 단어를 예측하게 된다.

기계학습은 이렇게 엄청난 반복을 통한 확률적인 원리로 이루어지므로 학습 알고리즘은 인간이 설계하지만, 실제로 학습이 일어나는 과정과 학습 데이터를 기반으로 예측할 때 거치는 구체적 과정들은 그저 학습 데이터에 끼워맞춘 가중치에 의해 일어나는 무작위적인 현상

¹⁸인공신경망의 경우와 마찬가지로 학습 알고리즘에 의해 결정된 의미나 특징의 기준들이 우리의 직관으로는 이해할 수 없을 가능성이 높다.

에 불과하다. 이러한 특성은 모델이 특정 상황에서 특정 예측을 내놓는 원인을 파악하는 것을 매우 어렵게 한다.

II.2. 트랜스포머

본래 트랜스포머는 구글에 근무하던 8명의 연구원이 쓴 “Attention Is All You Need”¹⁹라는 논문에서 처음 제안되었다. 제목은 비틀즈의 노래인 “All You Need Is Love”를 패러디한 것이며, 처음 제안된 트랜스포머 구조의 목적은 영어와 독일어 간 번역이라고 한다. 이 트랜스포머는 AI 업계에 엄청난 파급력을 행사했고 AI 붐을 일으켰다. 현재 우리가 쓰는 ChatGPT, Gemini, Claude, 딥시크, LLaMA, Grok 등의 챗봇 모델은 모두 트랜스포머 기반이다.

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.²⁰

— Attention Is All You Need (2017), Ashish Vaswani, et al.

¹⁹원문: <https://arxiv.org/abs/1706.03762>

²⁰현재 주류인 시퀀스 변환(sequence transduction) 모델들은 인코더-디코더 구조에서 복잡한 순환 신경망(RNN) 또는 합성곱 신경망(CNN)에 기반하고 있다. 현재 가장 우수한 성능을 내는 모델들은 또한 어텐션을 통해 인코더와 디코더를 연결한다. 우리는 ‘트랜스포머’라고 이름붙인 새로운 단순한 신경망 구조를 제안한다. 이는 전적으로 어텐션에만 기반하며, 순환(recurrence)과 합성곱(convolution)을 완전히 없앴다. 기계 번역 과제에 대한 두 개의 실험에서, 이 모델이 질적으로 더 우수할 뿐 아니라, 병렬화가 더 잘 되고 학습에 드는 시간도 현저히 적게 든다는 것을 보였다. 우리 모델은 WMT 2014 영어-독일어 번역 과제에서 BLEU 점수 28.4를 달성하였으며, 이는 (양상블 모델들을 포함한) 기존의 최고 성능 결과보다 2 BLEU 이상 향상된 수치이다. 또한 WMT 2014 영어-프랑스어 번역 과제에서는 단일 모델 기준으로 BLEU 점수 41.8을 달성하며 최고 성능 신기록을 갱신했다. 이 모델은 8개의 GPU로 3.5일 동안 학습됐으며, 이는 현존 문헌에 보고된 최고 모델들의 학습 비용의 극히 일부에 불과하다. 영어 구문 분석 과제에 적용했을 때 대규모 학습 데이터와 제한된 학습 데이터 모두에서 우수한 결과를 얻음을 근거로, 트랜스포머는 다른 일반적인 작업도 잘 수행할 수 있음을 밝힌다 (필자 의역).

이처럼 트랜스포머는 번역 작업 이외에도 음성인식, TTS, 그림 인식, 그림 생성, 일반적인 LLM 등의 역할을 범용적으로 수행할 수 있다. 트랜스포머의 작동원리를 좀 더 깊게 알아보자.

트랜스포머는 먼저 입력을 일정 단위로 쪼개는데, 이때 쪼개진 것들을 토큰이라고 한다. 토큰은 입력이 문자열일 때는 보통 단어이거나 일반적으로 자주 등장하는 문자 조합 따위이다. 사진이나 소리라면 입력을 일정 단위로 쪼개 샘플이 토큰이 된다. 이 토큰은 각각(여기서는 숫자의 배열의 의미에 가까운) 벡터로 할당되는데, 이 벡터들은 숫자로서 토큰과 관련된 모종의 의미를 나타내게 된다. 이 의미의 기준을 우리가 알 수는 없겠지만 이 벡터가 아주 고차원의 공간에 있다고 상상하면, 비슷한 의미를 가진 단어들은 공간 상에서의 위치가 비슷할 것이라고 이해할 수 있다.

다음으로는 트랜스포머의 핵심인 어텐션 과정을 거친다. 어텐션은 문장 속 각 단어들 간 상호작용을 통해 문장의 문맥 속에서 단어가 어떤 의미를 가지는지를 더욱 특정하고 조정한다. 어텐션 이후에는 의미의 벡터를 순전파 신경망(다중계층 퍼셉트론이라고도 한다)에 보내 의미를 구체화하고 고차원적으로 만드는데, 이 과정에 대해서는 나중에 더 자세히 알아보겠다. 이 모든 작업은 거대한 행렬과 벡터 간의 행렬곱의 형태로 나타난다.

어텐션과 순전파를 충분히 반복한다면 문장의 마지막 벡터에 문장 전체의 의미가 충분히 녹아들었다고 할 수 있고, 이때 그 마지막 벡터에 특정 연산을 수행해 가능한 모든 토큰 중 문장 다음에 올 것을 예측하는 확률분포를 도출할 수 있다. 여기서 가장 확률이 높은 것(또는 조금의 무작위성을 주어 두세번째로 높은 것)을 골라 문장에 넣고 이 전체 문장을 다시 입력하면 트랜스포머 모델이 재귀적으로 돌아가며 출력을 뽑아낸다.

이런 시스템을 챗봇으로 사용하는 가장 간단한 방법은, 미리 다음과 같은 프롬프트를 주고

아래는 사용자와 유능하고 매우 지능적인 AI 조수의 대화이다.

사용자: (a)

AI:

사용자의 입력을 (a) 부분에 넣은 뒤 LLM에게 저 글 전체를 던져주면 그 다음에 오는 출력이 곧 챗봇의 응답이 되는 것이다. 이 방식은 지금도 쓰이고 있다. 이 원리를 안다면 요즘 챗봇 서비스에서 제공하는 개인 설정 프롬프트를 작성할 때 어떻게 작성하면 효과적인지도 알 수 있을 것이다.

II.3. 단어 임베딩

토큰은 꼭 단어 단위로 나뉘는 것이 아니지만, 이해를 위해 단어 단위로 나눈다고 하겠다²¹. 트랜스포머 모델은 자기가 학습한 모든 단어의 의미²²를 행에 저장한 거대한 행렬을 하나 갖는데, 각 토큰에는 고유한 인덱스 ID가 붙어있어 이 행렬의 몇번째 행에서 단어의 의미를 찾아야하는지를 지시한다.

$$W_E \in \mathbb{R}^{V \times d} \quad (2.1)$$

이 행렬을 임베디드 행렬(embedded matrix)라고 하고, 보통 W_E 로 표시한다. V 는 학습한 어휘의 총 개수, d 는 의미 벡터의 차원 수이다. 이 행렬의 값들은 기계학습의 모든 다른 값들처럼 처음에는 무작위로 시작하지만 학습 과정에서 조율된다. 토큰을 입력받으면 임베디드 행렬에서 그 의미를 받아와 토큰에 초기값으로 할당하게 된다.

그런데, 단어가 숫자로 표현되는 원리는 뭔가? 이 방법은 사실 기계학습에서는 트랜스포머나 LLM의 등장 이전에도 존재했는데, 이것을 임베딩이라고 한다. 단어의 의미 벡터는 아주 많은 기저로 이루어진 고차원 공간 상에 있다. 이 공간 상에서 단어들은 반복된 학습을 통해 비슷한 의미를 가진 것끼리 비슷한 경향성을 띄게 되고, 특정 의미 기준이 특정한 방향을 의미하게 된다.

충분히 학습되었다고 할 때, 이 벡터들 간에는 흥미로운 관계가 있다. $Em(\text{단어})$ 를 “단어”에 대한 의미 벡터라고 하자. 이때 예를 들자면 아래와 같은 관계가 성립한다.

$$\begin{aligned} Em(\text{아버지}) - Em(\text{어머니}) &\approx Em(\text{남자}) - Em(\text{여자}) \\ &\approx Em(\text{이모부}) - Em(\text{이모}) \approx Em(\text{고모부}) - Em(\text{고모}) \approx \dots \end{aligned} \quad (2.2)$$

즉, “남자 부모”를 뜻하는 단어를 모를 때, 아래와 같이 그 단어(또는 비슷한 단어)를 알아낼 수 있다.

$$Em(\text{아버지}) \approx Em(\text{어머니}) + Em(\text{남자}) - Em(\text{여자}) \quad (2.3)$$

3Blue1Brown의 예시를 빌리자면, 이 관계는 단수-복수 관계에서도 성립한다. 예를 들어 복수를 뜻하는 plural의 p를 따

²¹문장부호, 어근과 접사, 어간과 어미, `< |endoftext| >` (GPT-3의 경우 응답을 끝낼 지점)도 모두 토큰이다.

²²이제부터 ‘의미’라고 하면 우리가 이해하는 음성적, 연상적 의미가 아니라 기계학습에서 벡터로 표시되는 수치적인 의미를 말한다.

$$\mathbf{p} := \text{Em}(\text{cats}) - \text{Em}(\text{cat}) \quad (2.4)$$

라고 한다면 다음과 같은 관계가 성립한다.

$$\mathbf{p} \cdot \text{Em}(\text{students}) \gg \mathbf{p} \cdot \text{Em}(\text{student}) \quad (2.5)$$

흥미로운 점은, 이것이 꼭 단수-복수 단어의 관계에서만 보이는 경향이 아니라는 것이다.

$$\mathbf{p} \cdot \text{Em}(\text{one}) \ll \mathbf{p} \cdot \text{Em}(\text{two}) < \mathbf{p} \cdot \text{Em}(\text{three}) \dots \quad (2.6)$$

이런 식으로, 비슷한 의미 기준은 벡터공간 상에서 같은 방향으로 정렬되어 있는 경향을 보인다. 사실 이 모든 것이 선형회귀의 원리의 연장선 상에 있다고 생각하면 이해가 편한 것 같다.

어텐션 같은 연산을 받는 경우, 임베딩은 꼭 단어의 의미에만 의존하는 것이 아니다. 예를 들어 문장에서 단어의 위치도 벡터를 조작할 수 있으며, 그 단어를 수식하는 단어에 의해 의미가 조작될 수도 있다. 이런 식으로 의미는 더욱 더 특정화, 구체화되고 풍부하게 변하게 된다.

모델이 글을 읽게 되면 먼저 임베딩 벡터에서 단순히 단어의 의미를 가져온다. 하지만 트랜스포머 모델의 목표는 각 단어가 개별적으로 의미하는 것보다 더 구체적이고 풍부한 의미를 얻는 것이므로 이를 위해 어텐션을 수행한다. 인공지능 모델은 (당연하게도) 정해진 수의 벡터로만 연산할 수 있는데, 이런걸 context size 또는 token size라고 한다. 이 연산에 대해서는 후술하겠다.

마지막에는 문장의 끝에 있는 의미 벡터를 전체 단어의 행렬로 투영하는 행렬 연산을 적용한다. 이렇게 출력된 벡터에 softmax라고 하는 정규화 연산을 적용해 합이 1인 확률분포를 만든다. 이때 행렬 연산으로 작용하는 행렬을 역임베딩 행렬(unembedding matrix)라고 하고 W_U 로 표기한다. 이 행렬의 값 또한 처음에는 무작위 값이었다가 학습을 통해 조정된다.

softmax 연산의 일반식은 아래와 같다.

$$x_k \leftarrow \frac{\exp(x_k)}{\sum_i \exp(x_i)} \quad (2.7)$$

즉 이것이 벡터에 작용하면 이런 모양이다.

$$\text{softmax} \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} = \begin{bmatrix} \frac{\exp(x_1)}{\sum_{i=1}^N \exp(x_i)} \\ \vdots \\ \frac{\exp(x_N)}{\sum_{i=1}^N \exp(x_i)} \end{bmatrix} \quad (2.8)$$

하지만 항상 가장 확률이 높은 것을 선택하는 것보다는 적당히 높은 것도 선택하는 것이 창의성, 다양성 같은 걸 증대할 수 있으므로, 온도²³라고 불리는 상수 T 를 추가해 무작위성을 줄 수 있다.

$$x_k \leftarrow \frac{\exp\left(\frac{x_k}{T}\right)}{\sum_i \exp\left(\frac{x_i}{T}\right)} \quad (2.9)$$

T 가 클수록 더 작은 값에도 더 큰 확률을 부여해 확률 값을 평준화시킨다. 이때 softmax 연산을 적용하기 전의 언임베딩 연산을 거친 마지막 토큰의 의미벡터를 logit이라고 하기도 한다.

II.4. 어텐션

앞서 보았던 ‘배’의 예시로 계속하자면, 아래 문장들에서 ‘배’의 의미는 모두 다름을 우리는 알 수 있다.

- 배(腹)가 아프다.
- 배(船)가 항구를 떠난다.
- 애플, 겨울 배(梨)가 맛있단다!
- 두 배(倍) 증가했다.

우리는 이 ‘배’의 의미를 문장 주변의 단어들로 알 수 있다. 이 예시에서는 주로 서술어로 그 의미를 유추할 수 있다. 하지만 언어 모델이 이런 문장들을 처음 입력받을 때, 토큰 ‘배’는 모두 같은 의미 $\text{Em}(\text{배})$ 로 표현된다. 어텐션은 이 배의 의미를 주변 토큰들과 상호 연산 시킴으로써 구체화시키는 역할을 한다.

또, 주어진 프롬프트로 그림을 생성하는 AI에게 아래 구를 입력했다고 해보자.

- 경비행기 프라모델

‘비행기’라는 말을 들으면 보통 대형 민항 여객기나 작은 구식 프로펠러기를 떠올리는 것이 일반적이다. 하지만 앞에 접사 ‘경(輕)-’이 붙으므로 소형 비행기(세스나 社の 경비행기가 시장

²³열역학에서 따온 비유이다. 열역학에서 온도 T 는 입자의 평균 운동에너지 $K = \frac{f}{2} k_B T$, 엔트로피(무질서도) $dS = \frac{\delta Q}{T}$ 등을 나타낸다.

에서 압도적이므로 보통 세스나-172 같은 모델이 생성된다)라는 의미가 특정된다. 뒤에 오는 프라모델이 이 모든 걸 수식받음으로써 실제 비행기가 아니라 작고 장난감같이 보여야한다는 정보가 추가된다. 보통 프라모델, 미니어처, 장난감 등의 말을 이런 거대한 물체에 대한 말에 추가로 붙이게 되면 AI는 물체에다가 냅다 키팅을 달아버리고 밝은 빛과 바닥이 있는 환경에 던져놓는다(이번 예시에서는 경비행기라는 물체가 너무 구체적이었는지 키팅을 달지는 않았다.).

Figure 2.1 — 비행기, 경비행기, 세스나 경비행기, 경비행기 프라모델 이미지 생성 결과
(Google Imagen 4 via Gemini 2.5 Flash assistant)



II.5. 임베딩과 어텐션의 작동 원리

이제 수학적으로 들어가 보자. 어텐션을 설명할 예시 문장으로 “애, 겨울 배가 맛있단다!”를 채택하도록 하겠다.

앞서 토큰은 단어라고 했지만 그건 영어 화자들에게 잘 먹혀들어가는 설명이다. 한국어, 일본어 같은 교착어나 독일어, 러시아어 같은 교착어는 어미와 접사의 역할이 중요하므로 단어 또는 어절 단위로 나누어지지 않는다. 굳이 말하면 (토큰나이저 알고리즘에 따라 다르겠지만) 토큰은 형태소에 가깝다고 하는게 좋을 것 같다. SentencePiece나 BPE 같은 토큰나이저를 사용한다면 아래와 같이 쪼갤 수 있다.

["애", ",", " ", "겨울", "배", "가", "맛", "있", "단", "다", "!", ""]

이제 이 단어들은 W_E 에 의해 의미 벡터로 대체된다. 예를 들어 ‘겨울’의 ID가 666이라고 하면

$$\text{Em}(\text{겨울}) = W_E[666] = \mathbf{e}_{\text{겨울}} \in \mathbb{R}^d \quad (2.10)$$

또, 트랜스포머는 입력을 병렬로 처리하기 때문에 토큰의 순서를 제대로 알 수 없는데, 어텐션을 위해서는 순서를 알아야 하므로 이를 위해 임베딩된 의미 벡터에 추가적으로 위치 정보를 추가하게 된다. 이 과정을 positional encoding이라고 한다. 위치 p (입력 상에서 토큰의

위치 인덱스)와 차원 $i(d/2 - 1$ 과 같은 꼴의 값)에 대해 이런 식으로 삼각함수를 이용하여 위치 정보를 표현한다.

$$\begin{aligned} PE(p, 2i) &= \sin\left(\frac{p}{10000^{2i/d}}\right) \\ PE(p, 2i + 1) &= \cos\left(\frac{p}{10000^{2i/d}}\right) \end{aligned} \quad (2.11)$$

이것은 원 논문에서 제시된 방법으로, PE에 삼각함수를 이용하는 이유에는 몇 가지가 있다. 첫번째로, 이 PE 정보 자체가 의미 벡터에 들어가야하기 때문에 미분이 가능해야 해서 단순 정수를 사용할 수 없다. 사인곡선은 $\mathbb{R}$ 에서 연속이고 미분가능할 뿐 아니라 파동으로서의 주기성과 위상 또한 가지고 있으며, 삼각함수의 덧셈정리를 이용해 비교적 용이하게 $\sin(a + b)$ 와 같은 선형결합을 계산할 수 있다. 즉, 토큰의 위치(p)가 변하면 삼각함수 값에 의해 벡터가 부드럽게 회전하므로 내재적으로 연속이면서 순서 정보를 함의한 값을 만들 수 있다. 또, 순서에 대한 별도의 추가적인 학습 없이 모델이 순서 “감각”을 갖출 수 있으므로 유용하다. 차원 i 는 최종 결과에서 삼각함수의 주기를 달리하여, 입력 내 토큰 간 거리 스케일에 따른 의미관계를 다양하게 표현할 수 있도록 한다. 즉, 하위 차원은 국소적인 간격에서의 세밀한 위치변화를, 상위 차원은 전체 입력 문장 구조에서의 긴 거리 간 위치변화를 담당한다.

설명이 난해한데, 정리하자면 PE를 삼각함수로 작성한 것은 토큰 간 순서, 상대적 간격, 문맥적 구조를 파동의 위상 회전이라는 내재적인 수학적 특성으로 표현하여 합, 내적 등 간단한 연산만으로 어순과 맥락을 쉽게 알아낼 수 있도록 하기 위한 것이다.

우리의 문장에서 ‘겨울’이 $p = 2$ 이므로²⁴ 예시 계산식으로 계산해보면

$$PE_{\text{겨울}} = \begin{bmatrix} 0.002 \\ 0.998 \\ \vdots \end{bmatrix} \quad (2.12)$$

같이 된다.

이제 최종 입력 벡터는 의미와 PE를 합친

$$\mathbf{x}_p = \mathbf{e}_{\text{token}} + PE_p \quad (2.13)$$

가 된다.

이런 식으로 의미벡터가 모인 전체 입력은 이런 식으로 생겼을 것이다.

²⁴인덱스는 0부터 센다.

$$X = [x_0, x_1, \dots, x_{n-1}] \quad (2.14)$$

각각의 입력 벡터들은 이제 세 개의 서로 다른 연산자와 결합해 선형변환을 받는다.

$$\begin{aligned} Q_i &= W_Q x_i \\ K_i &= W_K x_i \\ V_i &= W_V x_i \end{aligned} \quad (2.15)$$

W_Q, W_K, W_V 는 학습으로 값, 즉 구체적 역할이 정해지는 행렬들이고, Q_i 는 query로 어떤 정보를 찾고자 하는지에 대한 것이고, K_i 는 key로 어떤 정보를 가지고 있는가, V_i 는 value로 정보의 내용이다. 여기서 ‘정보’란 의미에 관한 정보로, 결론적으로 어떤 토큰이 의미 벡터 공간에서 어떤 식으로 다른 토큰에 변환을 주어야 하는가에 대한 것이다. query의 질문을 key가 답하는 것이라고 보면 된다.

예를 들어 ‘배’라는 토큰을 입력 내의 다른 토큰 ‘겨울’, ‘맛’ 등과 비교해 ‘배’의 의미와 가장 관련이 있어서 그 의미를 구체화할 수 있는 토큰을 찾아낸다. 이때 의미가 관련된 정도는 attention score로 결정되는데, $\text{score}(i, j)$ 는 토큰 $p = i$ 가 $p = j$ 에게 얼마나 주의(attention)를 기울이는가를 나타낸다. ‘attention’이라는 말이 여기서 나온 것이다.

$$\text{score}(i, j) = \frac{Q_i \cdot K_j}{\sqrt{d_k}} \quad (2.16)$$

여기서 $d_k = \dim(Q \cdot K)$ 로, 값이 폭주하는 것을 막는다. 이제 이 값에 softmax를 적용하여 합을 1로 정규화한다.

$$\alpha_{ij} = \text{softmax}(\text{score}(i, j)) \quad (2.17)$$

이제 각 토큰의 value 벡터인 V_j 를 정규화되어 확률분포로 볼 수 있는 가중치 α_{ij} 로 가중치 합하여 선형결합하면 주변 토큰의 문맥이 반영된 배의 의미가 도출된다.

$$z_i = \sum_j \alpha(ij) V_j \quad (2.18)$$

우리의 문장에서 원래 ‘배’는 그냥 ‘배’라는 단어의 일반적 의미를 가지고 있었으나, ‘맛있단다’ 쪽에 어텐션을 둔 후 그 값이 조정되어 최종적으로 과일로서의 배의 의미를 가지게 된다. 이런 과정들은 각 토큰에 대해 수행되는데, 이것을 행렬로 모아서 식을 간결히 할 수 있다.

$$\begin{aligned} Q &= XW_Q \\ K &= XW_K \\ V &= XW_V \end{aligned} \quad (2.19)$$

이제 전체 어텐션 연산이 한 줄로 표현된다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.20)$$

이것이 원 논문에서 제시된 어텐션 연산의 식으로, 논문의 핵심 내용이자 트랜스포머를 대변하는 식이라고 할 수 있다.

트랜스포머는 이 과정을 병렬로 한꺼번에 많이 수행한다. 즉 서로 다른 W_Q, W_K, W_V 를 가진 여러 개의 “헤드(head)”가 있는데²⁵, 각 헤드가 서로 다른 “관점”에서 각기 다른 기준으로 입력을 평가한다. 다시, 이 기준은 학습의 결과이기 때문에 우리가 상식적으로 이해하지 못할 수도 있다.

$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h) \quad (2.21)$$

이제 각 헤드의 결과를 결합하면

$$Z = \begin{bmatrix} z_i^{(1)} \\ z_i^{(2)} \\ \vdots \\ z_i^{(H)} \end{bmatrix} \quad (2.22)$$

처럼 되고, 이제 여러 헤드의 결과를 선형결합하면

$$\text{MultiHead}(Q, K, V) = \begin{bmatrix} \text{head}_1 \\ \vdots \\ \text{head}_H \end{bmatrix} W_O \quad (2.23)$$

여기서 W_O 는 출력 행렬로, 최종 의미 결합을 위해 학습된 또 다른 행렬이다.

이제 이 어텐션을 거친 의미들을 다중계층 퍼셉트론, 즉 아래와 같은 순전파 신경망에 통과시킨다.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2.24)$$

²⁵실제 구현에서는 각 헤드가 전체 행렬에서 헤드 수만큼 분할된 가중치를 공유한다고 한다.

신경망에서는 주로 ReLU인 활성화함수에 통과시킨다. 이때 미리 학습해놓은 데이터(주로 사실 관계 등)와 행렬곱(각 벡터에 대한 내적으로 볼 수 있다) 연산을 수행하여 현재의 의미가 사실관계와 얼마나 일치하는지를 확인하고 보정하는 과정이 일어난다는 추측이 있다

2627.

그 다음, 다시 어텐션을 하고 퍼셉트론에 보내는 것을 여러번 반복한다. 이때, 이미 지정된 문맥적 의미를 바탕으로 또 새로운 문맥적 의미가 추가되어 의미가 구체화되고 깊어지게 된다. 말하자면 단순히 문법적 구조를 파악하는 것을 벗어나, 문장의 의도가 무엇인지, 어떤 표현법이 사용되었는지, 정서는 어떤지 등등을 알아낼 수 있다고 기대하면서 여러번 이것을 반복하는 것이다.

II.6. 트랜스포머의 한계

트랜스포머가 작동하는 과정을 이해했다면 트랜스포머 모델의 한계도 체감할 수 있다. 첫째로, 기존에 존재하던 RNN 등의 방식보다는 연산이 간결한 것은 맞지만 여전히 $O(n^2)$ 으로 엄청나게 복잡하다. GPT-3의 경우 학습 시 조정되어야 할 어텐션 관련 가중치 파라미터는 자그마치 57,982,058,496 개²⁸이다. 또, 토큰 사이즈를 늘리면 늘릴수록 행렬들의 차원 수는 제곱으로 늘어나기 때문에 연산의 복잡도가 제곱으로 늘어난다. 따라서 학습과 사용 모두에 엄청난 컴퓨팅 자원이 요구된다. 또, AI 챗봇의 상황에서 대화가 길어지고 내용이 많아지면 챗봇이 대화 내용을 까먹게 되거나 왜곡하면서 갈수록 대답의 질이 떨어진다.

매우 얇은 지식으로 조심스럽게 유추해보건대, AI 회사들은 현재 새로운 모델을 발표할 때 혁신이라고 대차게 광고하면서 내부적으로는 이미 성능 향상에 기여한다고 알려진 조건들, 예를 들어 컴퓨팅 자원, 많은 학습량, 효율적인 학습 및 정보처리 알고리즘, 어텐션을 반복하는 횟수 등만 늘리는 것이 아닌가 한다. 우리가 GPT-3보다 GPT-5가 압도적으로 똑똑하다고 느끼는 것은 주로 학습량이 비교가 안 될 정도로 차이가 나기 때문일 것이다. “추론” 또는 “이성” 모델³⁰이라고 하는 것은 실제로 모델이 뭔가 다른 것을 하는게 아니라, 사용자의 입력

²⁶출처: <https://www.alignmentforum.org/posts/iGuwZTHWb6DFY3sKB/fact-finding-attempting-to-reverse-engineer-factual-recall>

²⁷기계학습의 작동 방식 설계의 아이디어는 비교적 간단해 이해하기 쉽지만 왜 특정 학습 결과나 출력이 나타나는지 알아내는 것은 매우 어렵다. LLM이 사실관계를 확인하는 방법도 설계가 아니라 “현상” 중 하나인데, 사실관계 확인이 어떻게 일어나는지 제어가 어렵다는 것은 사실이 아닌 것을 뱉어내는 것도 제어하기 어렵다는 것이다. 그래서 AI 할루시네이션을 통제하기가 어렵다.

²⁸출처: <https://arxiv.org/pdf/2005.14165>

²⁹GPT-3의 어텐션을 제외한 전체 파라미터 수는 어텐션의 그것의 3 배 정도로 약 1750억 개이며, 이후 모델들의 사양은 정확히 공개되지 않고 있으나 확실한 것은 파라미터 수가 정말 엄청나게 많다는 것이다.

³⁰reasoning, thinking, think deeper 따위의 문구

을 바탕으로 모델이 자기가 수행해야 할 작업 체크리스트같은 것³¹을 먼저 만든 후 그 단계별로 출력을 수행하는 것이다. 즉, 상술이다. 트랜스포머는 분명 혁명적이었지만 분명 근본적인 한계가 있으며 현재는 그 한계를 단순히 컴퓨팅 자원을 늘려나가고 하드웨어를 개선하거나 FlashAttention, MQA, RoPE 같이 어텐션의 효율성을 증가시키는 것 등으로만 해결하고 있다. 그래서 어떤 사람³²들은 강인공지능이나 범인공지능이 등장하기 위해서는 트랜스포머에서 탈피해야 한다고 주장하기도 한다.

(반 농담) 무엇보다, LLM은 겨울 배가 맛있다는 문장을, 유행을 잘 아는 사람이라면 당연히 뒤에 뭐가 이어져야 하는지 아는 인간과 달리 뒤에 아무 말이나 만들어서 붙인다³³.

- 인간: 애, 겨울 배가 맛있단다! 배가 달아.
- LLM(예시): 애, 겨울 배가 맛있단다! 달고 시원하지?

재치가 없다는 소리이다. AI는 아직은 농담도 상황에 따라 제대로 만들어서 하질 못한다.

II.7. 2부 참고자료 및 출처

- Ashish Vaswani, et al. (2017), “Attention Is All You Need”
- Tom B. Brown, et al. (2020), “Language Models are Few-Shot Learners”
- 3Blue1Brown의 Neural Networks 시리즈
- Neel Nanda, et al. (2023), “Fact Finding: Attempting to Reverse-Engineer Factual Recall on the Neuron Level”
- 비행기 이미지: Imagen 4 via Google Gemini Flash 2.5

³¹이 과정에 chain of thought 이라는 번지르르한 이름이 붙었으며, GPT-4o, Claude 3 Opus 등에서 사용된다고 각 회사가 공식 발표했다.

³²일부 연구자(Yann LeCun, Marcus 등)는 ‘트랜스포머는 패턴 인식에는 탁월하나 추론과 일반화에는 한계가 있다’고 평한다. 다만 OpenAI나 DeepMind 등은 트랜스포머와 확장된 메모리, 멀티모달 입력 만 드응로 충분히 강인공지능에 근접할 수 있다고 주장한다.

³³혹시 이게 뭔지 모르는 독자는 인터넷에 ‘배 먹어 배’나 ‘겨울배’ 따위를 검색해 보도록 한다.